UNIT TEST REPORT

Electronic Store Inventory Management System

Academic Software Engineering Testing Deliverable | Automated Verification Suite

1. Introduction

The Unit Test Report documents the automated testing performed on the Electronic Store Inventory Management System. The purpose of unit testing is to verify that individual functions, modules, data models, business calculation services, and application components operate strictly in accordance with their functional specifications. Automated tests ensure that components behave deterministically and resist regression across software modifications.

The testing suite exercises core operations including user authentication, product catalog management, point-of-sale billing, purchase order procurement, inventory replenishment, customer returns and refunds, supplier administration, business analytics reporting, and automated database backup routines. Testing was executed using the Pytest framework under Python 3.14.6 in an isolated local virtual environment.

2. Objectives of Unit Testing

- **Verify Component Correctness:** Validate the correctness of individual functions, mathematical calculations, and business services under nominal and boundary inputs.
- **Identify Errors & Anomalies:** Detect logic defects, boundary violations, off-by-one errors, and unexpected runtime exceptions early in development.
- **Validate Input Handling & Processing:** Ensure invalid inputs—such as negative prices, non-numeric quantities, overlong text, and reversed date ranges—are safely rejected.
- **Verify Authentication & Authorization:** Confirm credential verification, session security, and role-based access control (Admin, Manager, Staff) across all endpoints.
- **Check Database Operations & Transactions:** Validate atomic commits, entity relationship integrity, foreign key cascades, and automatic rollback on failure.
- **Ensure Robust Error Handling:** Verify that unauthorized requests, missing database records, and corrupted inputs trigger informative error responses without system failure.

3. Testing Environment

Automated testing was conducted within the local repository virtual environment using an ephemeral SQLite database to guarantee clean test isolation without persistent database contamination.

Component / Parameter	Specification / Version
Programming Language	Python 3.14.6 (64-bit)
Web Application Framework	Flask 3.1.0
Testing Framework	Pytest 9.1.1
Coverage Instrumentation	pytest-cov 7.1.0 / coverage.py 7.17.2
Database Engine	SQLite 3 (In-Memory & Ephemeral File Fixtures)
Object-Relational Mapping (ORM)	SQLAlchemy 2.0 / Flask-SQLAlchemy 3.1
Operating System	macOS and Windows (both were used)
Development Environment	Visual Studio Code / zsh Terminal
Virtual Environment Path	./venv (Isolated Project Virtualenv)
HTTP Test Client	Flask WSGI Test Client (app.test_client())

Security Libraries	Bcrypt, ItsDangerous (cryptographic signing)

4. Test Modules and Functionality

The automated test suite comprises 20 test modules under tests/ supported by a shared fixture harness in tests/conftest.py. The suite encompasses both isolated unit tests (verifying isolated model properties, security algorithms, and helper routines) and integration-style automated tests (evaluating HTTP route lifecycles, session cookies, and database state transitions).

Test Module Name	Tests	Classification	Functional Scope & Verified Behaviors
test_app.py	2	Integration / Route	App factory creation, home page 200 OK, startup exception handling when table creation fails.
test_app_init_coverage.py	5	Unit / Core	User loader invalid/non-int IDs, legacy schema migration guards, supporting tables creation failure handling.
test_auth.py	22	Unit / Route	Login/logout flows, valid/invalid password verification, user creation, role assignment, password reset request.
test_auth_routes_coverage.py	9	Route / Security	Already-authenticated redirects, expired token validation, account deactivation/reactivation, role edit routes.
test_backup_database.py	2	Integration / CLI	Automated SQLite database backup generation, timestamped backup file creation, missing database error handling.
test_backup_database_coverage.py	7	Unit / CLI	Non-SQLite backend rejection, in-memory DB error handling, relative path resolution, integrity check failure, CLI exit codes.
test_main_routes_coverage.py	7	Route / Service	Threshold parsing, low-stock filter toggle, product form errors, restock/return exceptions, non-ISO date handling.
test_models_and_services_coverage.py	5	Unit / Model	Purchase order model helpers, product string representation, loyalty discount fallbacks, stock thresholds, password length limits.
test_password_reset_email.py	3	Unit / Service	Mocked SMTP email delivery, missing server config handling, STARTTLS vs SSL socket transport negotiation.
test_products.py	99	Unit / Route	Product CRUD lifecycle, barcode generation, duplicate SKU rejection, price/stock validation, category binding, search & pagination.
test_purchase_orders.py	80	Unit / Route / Integration	PO workflow (Draft, Pending Approval, Ordered, Received, Cancelled, Rejected), line items calculation, partial receipt, inventory increment.
test_purchase_orders_coverage_expansion.py	6	Route / Business	Supplier order link handling, empty order line validation, dynamic item quantity adjustment, action confirmation prompts.
test_report_exports.py	7	Unit / Service	CSV and PDF generation engines for sales reports, inventory movements, revenue summaries, and low-stock replenishment tables.
test_revenue_report.py	12	Route / Business	Revenue computation, profit margins, manager/admin access gates, date window filtering, reversed date error detection.
test_sales.py	53	Unit / Route / Integration	POS checkout, stock auto-deduction, customer discounts, receipt printing, return processing, refund computation, restock ledger.
test_sales_report.py	10	Route / Business	Sales analytics views, manager/admin role enforcement, staff restriction, date range validation, empty state rendering.
test_security.py	6	Unit / Security	Bcrypt salt uniqueness, non-plaintext storage, password verification, invalid hash rejection, max length boundary checks.
test_stock_report.py	8	Route / Business	Stock level summaries, inventory valuation, stock movement history ledger, date range filtering, RBAC restrictions.
test_supplier_routes_coverage.py	1	Route / Security	Supplier detail view route, authorization enforcement, 404 response on nonexistent supplier identifier.

Test Module Name	Tests	Classification	Functional Scope & Verified Behaviors
test_suppliers.py	29	Unit / Route	Supplier CRUD, admin-only access gates, contact info validation, 500-char address limit enforcement, empty list handling.
Total Automated Test Cases	**373**	**Full Suite Coverage**	**100% of automated test modules executed successfully with zero failures.**

5. Test Execution Summary

The complete automated test suite was executed using Pytest in the project virtual environment. The empirical execution parameters and outcomes are detailed below:

Parameter / Metric	Empirical Value	Status / Evaluation
Total Automated Tests Executed	373	100.0% of suite
Tests Passed	373	100.0% Pass Rate
Tests Failed	0	Zero Failures
Execution Errors	0	Zero Errors
Skipped Tests	0	Zero Skipped
Execution Duration	120.60 seconds (~2m 00s)	Completed
Overall Result	**PASS**	**All 373 Tests Passed**

6. Test Results Analysis Across Functional Domains

The automated test suite provides extensive functional verification across all nine primary architectural domains. The table below presents representative test cases derived directly from the automated test suite, illustrating the verified test scenarios, expected results, actual execution results, and final pass statuses:

Domain & Test Function	Test Scenario	Expected Result	Actual Result	Status
Authentication test_correct_password_logs_in	Log in with valid credentials (test-staff / StaffPass123).	System authenticates user, establishes session, redirects to dashboard.	Session established; 302 redirect to dashboard.	**PASS**
Authentication test_wrong_password_is_rejected	Submit invalid password for existing user account.	Login rejected with invalid credentials flash error; session denied.	Login rejected; error message displayed.	**PASS**
Product Management test_create_product_success	Create product with valid name, category, price, stock, and SKU.	Product entity persisted with generated barcode; redirected to catalog.	Product saved successfully with barcode.	**PASS**
Product Validation test_create_product_rejects_negative_price	Submit new product form with negative price value (-15.00).	Validation blocks submission; flash error shown; no database record.	Submission blocked; validation error raised.	**PASS**
Sales & Checkout test_create_sale_decreases_stock	Execute point-of-sale checkout for 2 units of product with stock=10.	Sale recorded atomically; product inventory reduced to 8.	Sale completed; inventory updated to 8.	**PASS**
Sales Validation test_create_sale_blocks_insufficient_stock	Attempt to checkout quantity exceeding available inventory (e.g. qty 15 vs stock 5).	Checkout blocked with insufficient stock warning; transaction rolled back.	Checkout denied; database rolled back.	**PASS**
Purchase Orders test_purchase_order_full_lifecycle	Progress purchase order from Draft -> Pending -> Ordered -> Received.	Order status advances; upon receipt, product stock is incremented.	Status transitions verified; stock incremented.	**PASS**
Purchase Order Rules test_cancel_received_order_is_blocked	Attempt to cancel purchase order after goods have been Received.	System rejects cancellation; order remains Received with error notice.	Cancellation blocked; status unchanged.	**PASS**
Supplier Management test_admin_can_create_supplier	Create new supplier entity with full contact details as Administrator.	Supplier record persisted; available for purchase order binding.	Supplier record created successfully.	**PASS**

Domain & Test Function	Test Scenario	Expected Result	Actual Result	Status
Supplier RBAC test_suppliers_list_rejects_staff	Staff user attempts to access supplier administration routes.	Access blocked with 403 Forbidden or redirect; view restricted.	Staff access denied; 403 response returned.	**PASS**
Returns & Restocking test_return_records_refund_from_original_sale_price	Process customer return for previously completed sale item.	Refund amount calculated from sale unit price; stock incremented in ledger.	Refund computed correctly; stock restored.	**PASS**
Reporting & Analytics test_sales_report_requires_both_dates	Request sales report leaving start date field empty.	System prompts for missing date parameter; report generation halted.	Validation triggered; missing date rejected.	**PASS**
Database Backups test_create_backup_successfully	Execute automated database backup script via CLI interface.	Timestamped SQLite backup created; integrity check (PRAGMA integrity_check) passes.	Backup created; integrity check passed.	**PASS**
Security & Cryptography test_password_is_not_stored_as_plain_text	Create user and inspect stored password field in SQLite database.	Password is encrypted via Bcrypt with unique cryptographic salt; no plaintext.	Password verified as salted Bcrypt hash.	**PASS**

Note: The table above illustrates representative test cases across the core functional modules. It represents a verified sample of the complete 373-test automated suite.

7. Automated Testing Evidence

All 373 automated tests were executed using the Pytest CLI within the isolated repository virtual environment (./venv). Testing evidence was compiled directly from execution terminal output and coverage logging:

- **Terminal Execution Logs:** Pytest collected 373 test items from 20 test modules in 0.08s. Full suite execution executed in 120.60 seconds with zero test failures and zero errors (373 passed in 120.60s).
- **Testing Command:** ./venv/bin/python -m pytest -v --cov=app --cov=scripts
- **State Isolation Verification:** Database transactions were isolated using ephemeral SQLite memory/file fixtures and scoped session teardown, ensuring zero cross-test interference.
- **Absence of Fictional Claims:** Verification evidence is established strictly through automated test logs, assertion outcomes, and coverage instrumentation. No manual browser execution was independently performed.

8. Conclusion and Scope Limitations

Unit testing was successfully executed on the Electronic Store Inventory Management System using Pytest. The automated test suite confirmed that application functions across authentication, product cataloging, point-of-sale checkout, purchase orders, inventory restocking, returns, supplier management, and database backups operate deterministically and conform to technical requirements.

The final test execution resulted in **373 passed tests, zero failed tests, zero errors, and zero skipped tests**, yielding a 100.0% pass rate. This demonstrates strong functional correctness and regression resistance.

Scope Limitations: While the automated test suite provides extensive functional verification, certain operational boundaries must be acknowledged:

- **External Network Relays:** Outbound email delivery (e.g. password reset links) is verified through mocked SMTP socket transport rather than live internet mail servers.
- **Client-Side Dynamic Execution:** Automated tests validate server-rendered Jinja2 templates and HTTP response payloads; client-side browser JavaScript execution and visual CSS layout are not evaluated.
- **High-Concurrency Load:** Tests run in isolated test client sessions; production database locking and performance under 50+ simultaneous concurrent users require dedicated multi-client load testing tools.